IntentFlow

AI SEO Intelligence Platform

PROJECT REPORT

Complete Technical Reference — System Design, OOP, Data Flow & Architecture

2026

IntentFlow: Comprehensive Project Report

1. Executive Summary

IntentFlow is a state-of-the-art AI SEO Platform designed to bridge the visibility gap in the evolving landscape of AI-driven search. As users transition from traditional search engines like Google to AI chat providers (ChatGPT, Claude, Perplexity), SEO strategy must evolve to capture user intent from these conversational streams. IntentFlow provides a comprehensive monorepo ecosystem that captures, materializes, and analyzes AI search behavior into deterministic, actionable campaign data.

2. Technical Architecture

2.1 Backend Materialization Engine

The backend is a high-performance Node.js/Express service utilizing Prisma ORM with PostgreSQL. Its primary responsibility is the materialization of raw AI stream events into a hierarchical tree structure (Prompt -> Subquery -> Site/Evidence). It features a robust JWT-based RBAC system and uses BullMQ for distributed job orchestration.

2.2 Extension Interception Layer

A Manifest V3 Chrome extension serves as the primary data ingestion point. It intercepts live AI provider streams (ChatGPT, Claude, Perplexity) using MAIN-world content scripts and forwards normalized data to the backend. It also includes a side-panel UI for real-time visualization and campaign management.

2.3 Operator Web Dashboard

Built with React 19 and Vite, the web dashboard offers advanced data visualization using XYFlow for campaign trees and Recharts for engagement analytics. It provides administrative controls for user management, event tracking, and lead intelligence scoring.

3. Team Members & Detailed Contributions

Sibtain Ahmed Qureshi - Project Lead & Architect

- Core Architecture: Designed and implemented the monorepo structure, ensuring seamless interaction between backend, web, and extension surfaces.

- Materialization Engine: Developed the deterministic mapping logic that converts unstructured AI responses into structured relational data.

- Database Infrastructure: Architected the Prisma schema and led the migration to PostgreSQL for enterprise-grade scalability.

Abhishek Verma - Strategy & Systems Design

- Lead Intelligence Framework: Conceptualized and designed the lead intelligence module, defining scoring models and behavioral signal capture.

- System Governance: Established the project's 'Agent Context' and governance rules, enabling scalable AI-assisted development across the team.

- Security & RBAC: Designed the multi-tenant architecture and granular RBAC (Role-Based Access Control) framework for the platform.

Amogha Raj Sandur - DevOps & Quality Assurance

- Project Governance: Managed the Task workspace and CI/CD alignment, ensuring that implementation tasks are mapped to strategic milestones.

- Code Quality: Led refactoring efforts across the monorepo to maintain high modularity and technical debt reduction.

- Documentation: Architected the technical handoff documentation (context.md) and established standards for task-based progress tracking.

Mohammed Yaseen - Full-Stack Engineering

- Mock Ecosystem: Built the comprehensive Mock Server and testing environment for local-first development without provider dependencies.

- Integration & Sync: Led the cross-surface synchronization efforts, ensuring data consistency between the Extension side-panel and Web Dashboard.

4. Future Scope & Roadmap

4.1 Server-Driven Replay & Automation

Transitioning from extension-based replay orchestration to a fully server-side execution model. This will allow for scheduled, background campaign refiring and monitoring without requiring active browser sessions.

4.2 Advanced Lead Scoring & CRM Integration

Implementing sophisticated machine learning models to refine lead scores based on historical interaction data. Planned automated exports to CRM platforms like Salesforce and HubSpot via custom webhook pipelines.

4.3 Enterprise Observability & Scaling

Integrating OpenTelemetry for end-to-end observability of data capture events. Scaling the Redis-based job queue to handle thousands of concurrent ingestion streams from enterprise pilot users.

4.4 Pilot Readiness & Commercial Launch

Finalizing the multi-tenant billing infrastructure and hardening security protocols for a public pilot launch. Focusing on SOC2 compliance readiness and high-availability deployment strategies.

1. What Does IntentFlow Do?

IntentFlow is a full-stack AI SEO Intelligence Platform. Its core purpose: capture what people type into AI chat tools (ChatGPT, Claude, Perplexity, Grok) and convert that hidden user behavior into structured, actionable SEO campaign data.

The Problem It Solves

Traditional SEO tools monitor Google searches. But millions of people now search via AI chatbots — and those queries are invisible to standard analytics. IntentFlow makes that invisible data visible.

The Core Workflow (End-to-End)

Step	What Happens	Component
1. User types in ChatGPT	Extension intercepts the live response stream in real-time	Chrome Extension (Content Script)
2. Stream is parsed	Prompts, search queries, and cited websites are extracted from raw token stream	Extension (Main World Script)
3. Data sent to backend	Structured JSON payload sent via API with auth token	Extension → Backend API
4. Turn is ingested	Backend normalizes and materializes the data into a tree of nodes	CampaignService.ingestTurn()
5. Campaign tree built	Prompt → Subquery → Site hierarchy stored in PostgreSQL	PromptNode table (Prisma)
6. Enrichment queued	SEMrush/Ahrefs jobs queued for keyword volume & traffic data	BullMQ Queue + Redis
7. Lead signals extracted	AI scans prompt for intent signals (buying intent, product interest)	LeadIntelligenceService
8. Dashboard shows data	Operator opens web app to see campaign tree, analytics, keyword data	React Web Dashboard

Key Features

- Multi-Provider Interception — Captures data from ChatGPT, Claude, Perplexity, and Grok simultaneously
- Deterministic Tree Mapping — Every AI conversation is materialized into a Prompt → Subquery → Site tree with stable, reproducible hash IDs

- Campaign Versioning — Campaigns can be snapshotted and 'refired' (re-run) to track how AI results change over time
- SEMrush & Ahrefs Integration — Background queues enrich site data with keyword volumes, traffic estimates, and rankings
- Lead Intelligence — Extracts buying signals from prompt text and scores users by intent level
- Admin RBAC Dashboard — Role-based access control with full admin visibility into users, events, and signals
- AI Prompt Suggestions — Uses OpenAI GPT to suggest follow-up prompts based on campaign history

2. System Architecture & Design

High-Level Architecture

IntentFlow is a monorepo with three deployable surfaces communicating via REST APIs and a shared PostgreSQL database.

> **Architecture Pattern**
> Monorepo with three independently deployable services: Backend (API server), Web (React SPA), and Extension (Chrome MV3). They communicate over HTTPS using JWT authentication.

Architecture Diagram (Text Representation)

CHROME EXTENSION	BACKEND (Node.js)	WEB DASHBOARD (React)
• Content Scripts (inject into ChatGPT/Claude etc.) • Background Service Worker • Side Panel React UI • Auth Token Storage	• Express.js REST API • Prisma ORM + PostgreSQL • BullMQ + Redis queues • JWT Auth middleware • Campaign tree engine	• Campaign graph view (XYFlow) • Analytics charts (Recharts) • Admin dashboard (RBAC) • Onboarding flow
React + MV3 + CRXJS	*TypeScript + Express + Prisma*	*React 19 + Vite + Tailwind*

Technology Stack

Layer	Technology
Backend Runtime	Node.js 20+ with TypeScript
Web Framework	Express.js 4
Database	PostgreSQL via Prisma ORM
Queue / Jobs	BullMQ on Redis
Authentication	JWT (access tokens) + Refresh tokens (stored in DB)
Frontend	React 19, Vite, Tailwind CSS, Shadcn UI / Radix UI
Graph Visualization	XYFlow (campaign tree diagram)
Charts	Recharts
Extension Build	Chrome MV3, CRXJS plugin, Vite
Validation	Zod (DTO schema validation)
External APIs	OpenAI GPT, SEMrush, Ahrefs
Package Manager	pnpm (workspace-aware)

3. System Design Principles Used

3.1 Layered Architecture (N-Tier)

The backend is strictly layered into 4 tiers. Each layer only communicates with the one directly below it. This enforces Separation of Concerns.

Layer	Files / Classes
1. Routes (HTTP)	campaign.routes.ts, auth.routes.ts — defines URL endpoints, applies middleware
2. Controller	CampaignController, AuthController — handles HTTP req/res, calls service
3. Service	CampaignService, AuthService — contains ALL business logic, orchestrates operations
4. Repository	CampaignRepository, AuthRepository — only layer that touches the database (Prisma)

> **Why This Matters**
> If you want to change the database from Postgres to MongoDB, you only change the Repository layer. The Controller and Service layers are untouched. This is called Low Coupling.

3.2 Monorepo Design

All three apps (backend, web, extension) live in one Git repository. This means shared type definitions can be kept consistent, and changes to the API contract are visible across the whole codebase at once. pnpm workspaces manage dependencies for each sub-project independently.

3.3 Multi-Tenancy

Every database record has a tenant_id column. A 'tenant' is essentially an organization or account. When user Alice at Company A creates campaigns, they are scoped to Company A's tenant_id. User Bob at Company B cannot see Alice's data even if both use the same backend server. This is called Row-Level Security via application code.

> **Tenant Isolation Pattern**
> Every service method accepts tenant_id as its first parameter. Every database query includes WHERE tenant_id = ? — enforced in the Repository layer. No tenant can ever access another tenant's data.

3.4 Event-Driven / Queue-Based Processing

When data is ingested from the extension, heavy enrichment tasks (calling SEMrush API, Ahrefs API, lead signal extraction) are NOT done synchronously. They are pushed into BullMQ queues backed by Redis. Worker processes consume these jobs asynchronously. This is the Producer-Consumer pattern.

- Producer: CampaignService.ingestTurn() pushes a job and returns immediately
- Queue: Redis via BullMQ stores the jobs
- Consumer: semrush.worker.ts, ahrefs.worker.ts process jobs in the background
- Benefit: The HTTP API responds in milliseconds; enrichment happens in parallel

3.5 Idempotency

The ingestTurn() function checks if a turn with the same request_id and turn_exchange_id already exists before writing. If it does, it returns the existing tree instead of creating a duplicate. This handles the real-world case where the extension might fire the same event twice due to network retries.

3.6 Deterministic Hashing

Instead of random UUIDs for internal reference IDs (prompt_ref, subquery_ref, result_ref), the system uses SHA-1 hashes of the content. This means the same prompt from the same provider always produces the same hash. This enables deduplication across multiple ingestion runs.

Example: `deterministic_hash(provider, turn_id, query_key, site_name, url)` → always the same result_ref for the same site in the same context.

3.7 Caching Strategy

SEMrush/Ahrefs results are cached in the SemrushSnapshot database table with a 24-hour TTL. Before calling the external API, the system checks if a recent snapshot exists. This prevents expensive repeated API calls for the same site.

3.8 Role-Based Access Control (RBAC)

Two types of roles exist. First, App Role: admin vs user — admins can see all tenants and system-level data. Second, Tenant Role: owner vs member — controls what you can do within your own organization. Middleware enforces this on every protected route.

Middleware	What It Checks
authMiddleware	Verifies JWT token is valid and not expired
requireRole('admin')	Checks app_role === 'admin' in JWT payload
requireAccountRole('owner')	Checks tenant_role === 'owner' for account-level operations

3.9 DTO Validation (Data Transfer Objects)

Every incoming API request body is validated using Zod schemas before reaching the Controller. This is defined in dto/ folders under each module. If the request body is missing required fields or has wrong types, a 400 Bad Request is returned immediately.

Example: `const data = createCampaignSchema.parse(req.body)` — throws if invalid.

3.10 Content Security Policy Workaround (Extension)

AI providers like ChatGPT have strict Content Security Policies that block inline script injection. IntentFlow solves this using Chrome Manifest V3's world:MAIN feature — the main-world script (chatgptStreamMain.ts) is declared in manifest.json and injected by Chrome itself, bypassing CSP. The isolated-world content script (chatgptStreamContent.ts) then listens for postMessage events from the main world and relays them to the extension background via chrome.runtime ports.

4. Object-Oriented Programming Concepts

IntentFlow is written in TypeScript which supports full OOP. Here are the key OOP patterns present in the codebase.

4.1 Abstraction — Abstract Base Classes

Three abstract base classes define the structural contract for the entire backend:

Abstract Class	File	Subclasses
BaseController	`src/core/controller.ts`	CampaignController, AuthController, UserController, etc.
BaseService	`src/core/service.ts`	CampaignService, AuthService, UserService, etc.
BaseRepository	`src/core/repository.ts`	CampaignRepository, AuthRepository, UserRepository, etc.

> **What is Abstraction Here?**
> The abstract base classes hide the internal complexity of each module. A developer extending BaseService knows they're building a service without needing to know how every other service works. The abstract class enforces a common identity.

4.2 Encapsulation — Private Methods in CampaignService

CampaignService has multiple private methods that hide internal logic from outside callers. These are marked with the private keyword:

- private resolveVersion() — internal logic to find or create the active campaign version
- private toVersionSummary() — converts a raw DB record to a clean response shape
- private toChatThreadSummary() — transforms a session into a summary object
- private toPromptCandidate() — maps a PromptNode to the public-facing PromptCandidateResponse type
- private listPromptCandidatesForVersion() — internal query hidden from the controller
- private listExecutedPromptsForVersion() — complex grouping/deduplication logic, hidden

The public API of CampaignService is simple (ingestTurn, createCampaign, getActiveTree, etc.) but the implementation complexity is encapsulated inside private methods.

4.3 Inheritance

All concrete classes inherit from their respective base classes:

`export class CampaignService extends BaseService { ... }` — inherits from BaseService

`export class CampaignController extends BaseController { ... }` — inherits from BaseController (implied)

`export class CampaignRepository extends BaseRepository { ... }` — inherits from BaseRepository

While TypeScript can achieve this via interfaces, the project uses abstract class inheritance which also allows shared utility methods to be added to the base class later without touching all subclasses.

4.4 Interfaces — TypeScript Structural Typing

Multiple interfaces define strict data shapes throughout the codebase:

- ApiSuccessResponse<T> and ApiErrorResponse — define the standard API response envelope
- CanonicalNodeMetadata — defines the precise shape of metadata stored per tree node
- MaterializedSubquery — internal interface for the tree-building algorithm
- WorkflowStatePayload — defines what getWorkflowState() returns
- ExecutedPromptResponse, PromptCandidateResponse — public response types

4.5 Polymorphism

The ApiResponse class demonstrates method-level polymorphism via generic types:

`static success<T>(data: T, message?: string): ApiSuccessResponse<T>` — the same method returns different types depending on T.

The provider stream system also demonstrates runtime polymorphism — the same streaming interface is used for ChatGPT, Claude, Perplexity, and Grok, with each provider's content script providing its own implementation of the stream interception logic.

4.6 Singleton Pattern

Services, controllers, and repositories are instantiated once and exported as singletons. This prevents redundant DB connections and ensures consistent state:

`export const campaignService = new CampaignService(campaignRepository);` — created once at module load.

`export const campaignController = new CampaignController();` — same pattern.

4.7 Dependency Injection

CampaignService receives its dependency (the repository) via its constructor rather than creating it internally. This is Constructor Injection:

`constructor(private repo: CampaignRepository) { }` — the repo is injected, not hardcoded.

This makes CampaignService independently testable — in a unit test, you can inject a mock repository and test the service logic in isolation.

4.8 Factory Pattern (ApiResponse)

The ApiResponse class acts as a factory — it creates standardized response objects. Instead of every controller manually building {success: true, data: ...}, they call:

`ApiResponse.success(campaign, 'Campaign created')` — which constructs the envelope consistently.

4.9 Strategy Pattern (Stream Providers)

The extension has a separate content script for each AI provider (chatgptStreamContent.ts, claudeStreamContent.ts, perplexityStreamContent.ts, grokStreamContent.ts). Each one implements the same event-posting contract (sends AI_SEO_* typed events) but uses a different interception strategy per provider's DOM/network behavior. The background.ts acts as the context that delegates to whichever strategy is active.

5. Database Design

The database is PostgreSQL managed through Prisma ORM. Here is the complete entity relationship structure.

5.1 Core Entities & Their Purpose

Table	Purpose
User	Platform user. Stores email, password hash, lead scoring data, app role
Tenant	Organisation/account. Every user belongs to one or more Tenants via TenantMember
TenantMember	Join table: User ↔ Tenant with a role (owner/member)
Domain	A website domain being tracked for SEO. Has scrape status and context JSON
DomainContext	Scraped content and page data for a domain (stored as JSON)
Campaign	An SEO campaign. Linked to a Tenant and optionally a Domain
CampaignVersion	Snapshot of a campaign at a point in time. Can be archived and refired
CaptureSession	One AI chat conversation (ChatGPT session). Linked to a CampaignVersion
CaptureTurn	One prompt-response exchange within a session. Stores raw_event_json
PromptNode	A node in the campaign tree. Can be type: prompt, subquery, site, or generated
SemrushSnapshot	Cached keyword data from SEMrush or Ahrefs for a site (24h TTL)
GenerationRun	Tracks an AI suggestion generation job (running/completed/failed)
LeadSignal	A lead intelligence signal extracted from a capture turn
AnalyticsEvent	User interaction events (for admin analytics views)
RefreshToken	Hashed refresh tokens for rotating JWT authentication
AuthCode	Short-lived auth codes for extension OAuth handshake

5.2 The Campaign Tree (Core Data Model)

The most important data structure is the PromptNode tree. It is a self-referencing tree where each node has a parent_id pointing to another PromptNode:

Node Type	Example content	Meaning
prompt (root)	"best SEO tools 2026"	The text the user typed into the AI chatbot. parent_id = null (root node)
subquery (child of prompt)	"top seo software"	A search query the AI ran internally. parent_id = prompt node ID
site (child of subquery)	ahrefs.com	A website cited in the AI's response. parent_id = subquery node ID
generated (sibling of prompt)	"compare seo pricing"	An AI-suggested follow-up prompt. source = 'suggestion' or 'manual' in metadata JSON

> **Self-Referencing Tree**
> PromptNode has a parent_id field that references another PromptNode.id. This creates a recursive tree structure in a single flat database table — a classic Adjacency List pattern.

5.3 Key Database Relationships

- User → Tenant: Many-to-many via TenantMember
- Tenant → Campaign: One-to-many (one org has many campaigns)
- Campaign → CampaignVersion: One-to-many (one campaign has multiple version snapshots)
- CampaignVersion → CaptureSession: One-to-many (one version has many chat conversations)
- CaptureSession → CaptureTurn: One-to-many (one conversation has many prompt-response turns)
- CampaignVersion → PromptNode: One-to-many (all tree nodes belong to a version)
- PromptNode → PromptNode: Self-referencing (parent_id, the tree structure)

5.4 Database Enums

The schema uses PostgreSQL enums for type safety:

Enum	Values
NodeType	prompt, subquery, site, generated
AiChatProvider	chatgpt, claude, gemini, perplexity, grok, unknown
CampaignVersionStatus	draft, active, archived
RunStatus	queued, processing, completed, failed
DomainScrapeStatus	queued, running, completed, failed
AppRole	admin, user
CaptureStatus	listening, intercepting, analyzing, generating, complete, failed

6. Data Flow — Detailed

6.1 Full Data Flow: Extension → Backend → Database

1. User opens ChatGPT in Chrome. Extension content script (chatgptStreamMain.ts) is injected into the ChatGPT page by Chrome (world: MAIN in manifest.json).
1. User types a prompt. The main-world script intercepts the network stream (XHR/Fetch) and extracts: the prompt text, any search queries ChatGPT ran, websites cited in the response.
2. Main-world script posts a message to the isolated-world content script via window.postMessage(). The isolated content script (chatgptStreamContent.ts) relays it to the background service worker via chrome.runtime.connect() ports.
3. Background service worker (background.ts) batches the events and calls the backend API: POST /campaigns/:id/ingest-turn with the structured payload including chat_provider, conversation_id, prompt, queries[], result_groups[].
4. Backend authMiddleware verifies the JWT token. The request reaches CampaignController.ingestTurn().
5. Controller calls CampaignService.ingestTurn(tenant_id, user_id, campaign_id, data).
6. Service normalizes the prompt text (clamps length, trims whitespace). Normalizes queries. Calls resolveVersion() to find or create the active CampaignVersion.
7. Service calls findOrCreateSession() in the Repository — finds or creates a CaptureSession for this conversation_id.
8. Service calls normalize_turn_events() — converts raw data into a sequence of typed events (prompt_sent, search_queries, search_results, response_finished) with deterministic SHA-1 hash IDs for every reference.
9. Service calls materialize_turn_from_events() — converts events into a structured object: { prompt, subqueries: [{ query_key, sites: [...] }] }.
10. Service writes a CaptureTurn record to DB with the full raw_event_json.
11. Service creates PromptNode records: first the prompt node (root), then subquery nodes (depth+1), then site nodes (depth+2). All are linked via parent_id.
12. Service fires leadIntelligenceService.extractSignalsFromTurn() as fire-and-forget (async, doesn't block response).
13. Service returns the current campaign tree (getActiveTree) to the controller.
14. Controller returns HTTP 201 with the tree JSON. Extension side-panel updates in real time.

6.2 Campaign Version & Refiring

Campaigns support versioning for tracking changes over time:

- When first data is ingested, establishActiveVersion() creates CampaignVersion v1
- User can 'refire' a campaign — this creates a new CampaignVersion (v2) as a copy/fork
- The new version starts fresh; re-running the same prompts in the AI tool will populate v2
- Users can compare v1 vs v2 to see how AI-cited sites have changed

6.3 Stream Interception Deep Dive

Each AI provider requires a different interception technique because they each deliver responses differently:

Provider	Interception Method
ChatGPT	Intercepts Server-Sent Events (SSE) stream from api.openai.com. Parses JSON chunks for search queries and citations.
Claude (claude.ai)	Intercepts streaming HTTP responses. Parses Claude's specific response format for cited sources.
Perplexity	Intercepts Perplexity's streaming API. Extracts 'web_results' from the response payload.
Grok (xAI)	Intercepts Grok's streaming responses and extracts search context.

6.4 Analytics Data Flow

The analytics system aggregates across all campaigns' captured data:

- getDashboardAnalytics(): Aggregates all CaptureTurn records by day, provider, and campaign. Returns momentum charts, freshness buckets, and health matrix.
- getPipelineAnalytics(): For a single campaign version — shows the funnel: suggested prompts → selected → fired → completed. Also shows execution heatmap by day/hour.
- getPromptAnalytics(): Per-prompt performance — keyword count, website count, success/failure by provider.
- getWebsiteAnalytics(): Site-level data — how many prompts discovered each site, aggregated SEMrush volume/traffic data, freshness of keyword data.